JECRC™
UNIVERSITY
BUILD YOUR WORLD

Department of Engineering

Session 2026-27

PROMPT ENGINEERING

(Course Code: BCO610A)

HW-05: THE HONEST REPOSITORY

GitHub & AI-Generated Documentation

Submitted By:

Name –Ankit Kumar Giri

Enrollment No. – 25BCON1276

Branch – B.Tech. CSE (AI&DS)

Semester – 3

Submitted To:

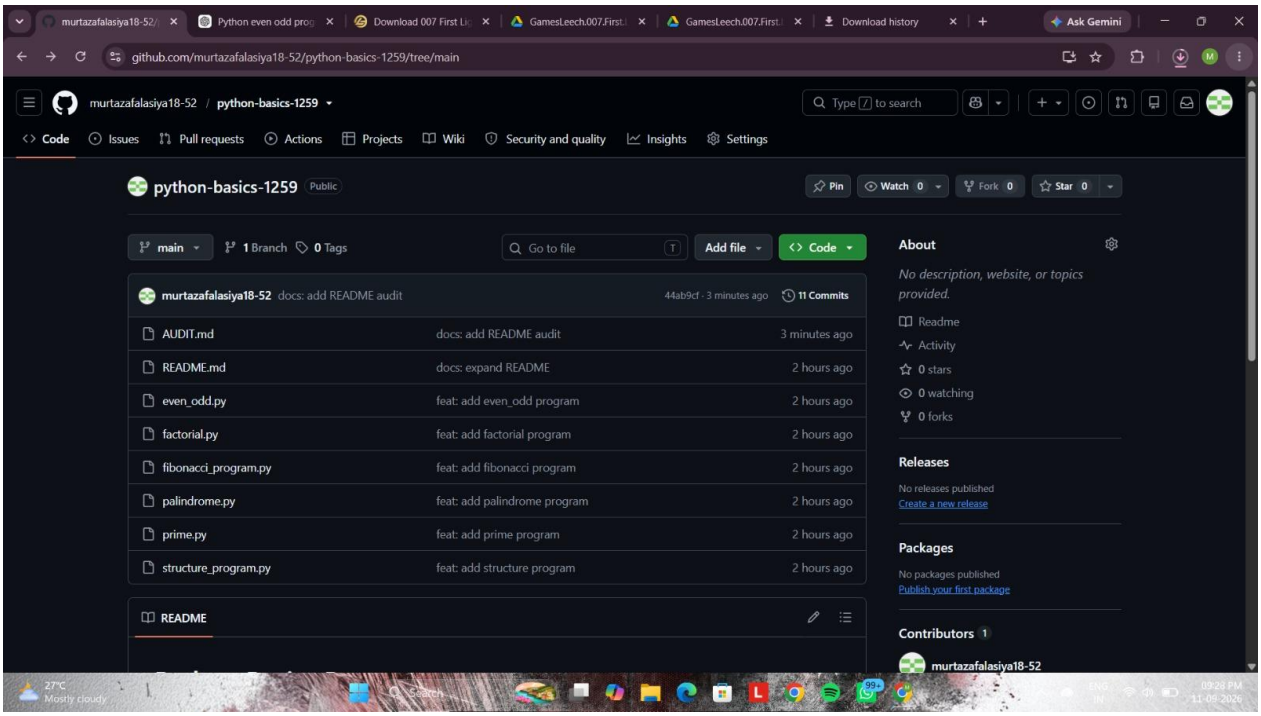
Mr. Ajay Sharma

Prompt Engineering

1. Public Repository + Repository Hygiene

Repository Details

Repository name: python-basics-25BCON1259
Visibility: Public
Repository link: https://github.com/murtazafalasiya18-52/python-basics-1259



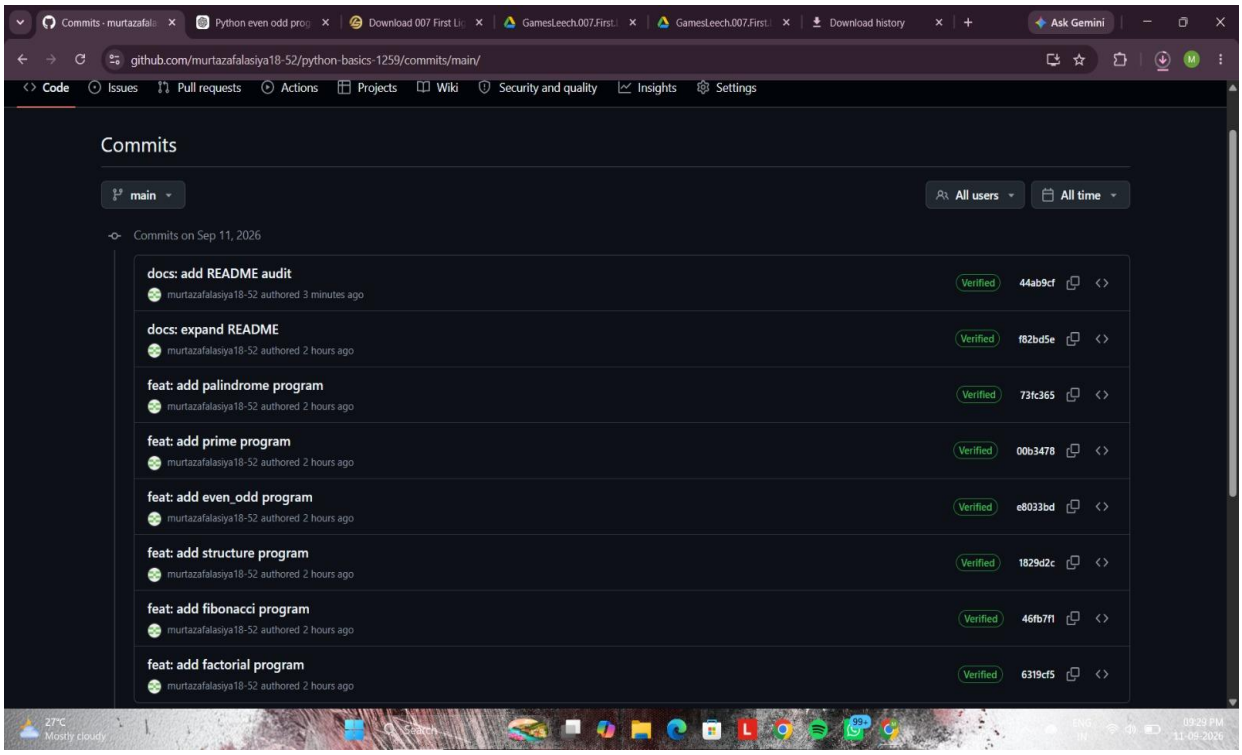
Six Programs

No.	Program / File	Purpose
1	factorial.py	Calculate factorial of a given number.
2	fibonacci.py	Generate a Fibonacci series.
3	structure.py	Store and display student information.
4	even_odd.py	Check whether a number is even or odd.
5	prime.py	Check whether a number is prime.
6	palindrome.py	Check whether a string/number is a palindrome.

2. Commit History + AI Message Comparison

My Commit Messages

Commit	My message	Change represented
1	feat: add factorial program	Added factorial.py
2	feat: add fibonacci program	Added fibonacci.py
3	feat: add student structure	Added structure.py
4	feat: add even odd checker	Added even_odd.py
5	feat: add prime checker	Added prime.py
6	feat: add palindrome checker	Added palindrome.py

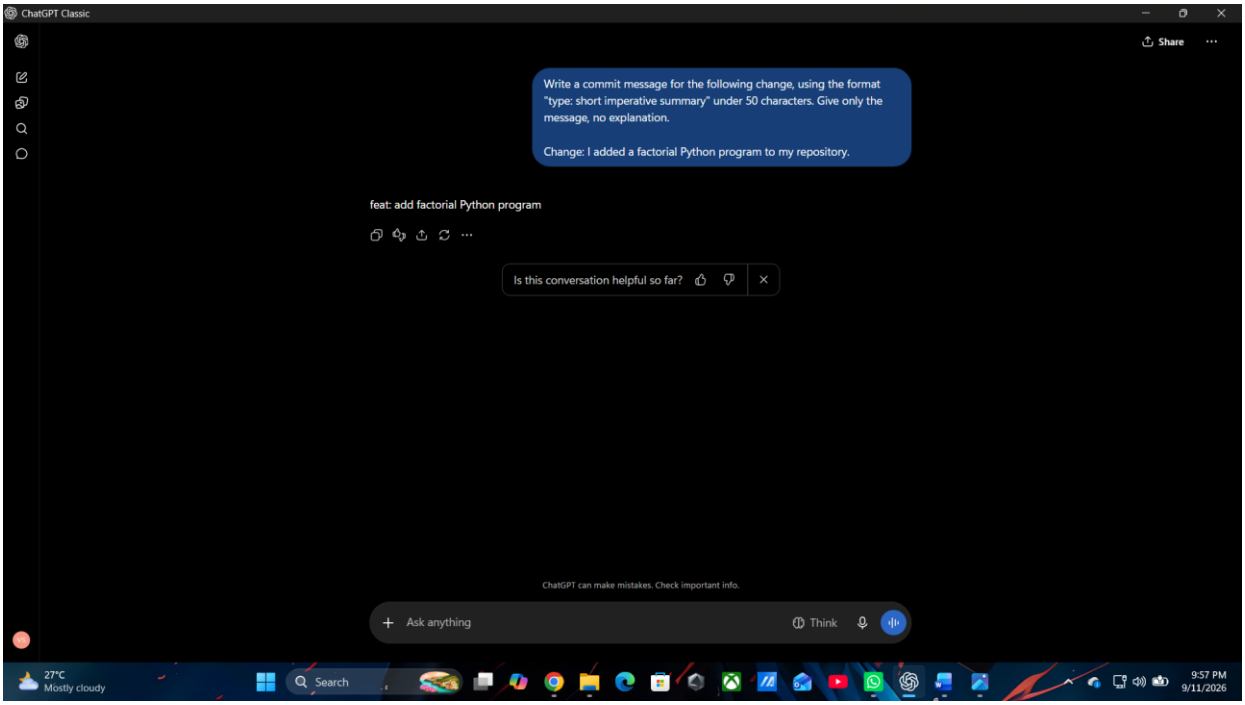


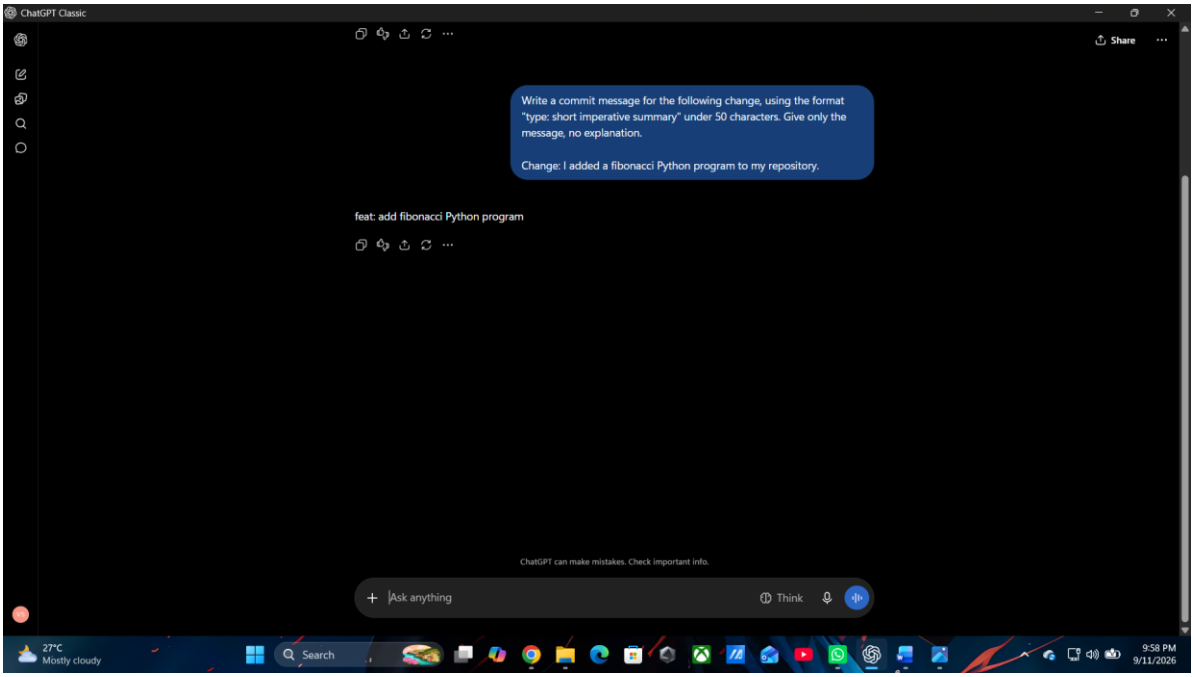
AI Commit-Message Prompt

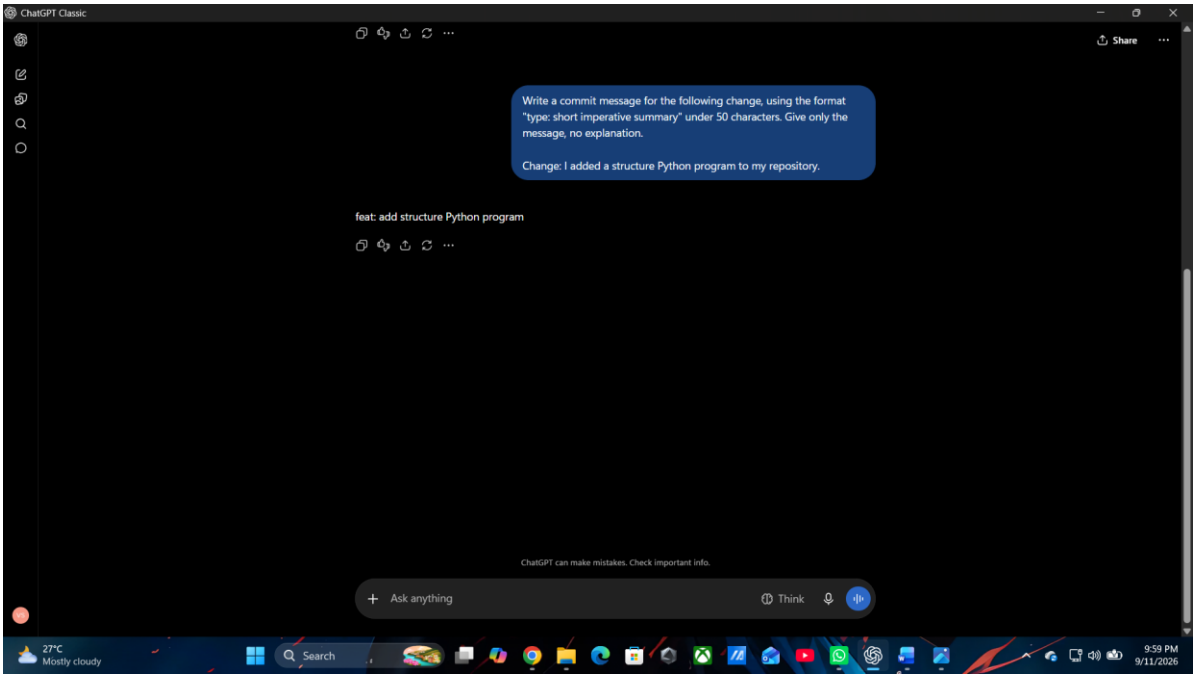
```
Write a commit message for the following change, using the format "type: short imperative summary" under 50 characters. Give only the message, no explanation.
```

Commit Comparison

Commit	My message	AI message	Which is clearer, and why?
1	feat: add factorial program	feat: add factorial progra	Both are equally clear because they describe the addition of the factorial program directly.
2	feat: add fibonacci program	feat: add fibonacci program	Both are equally clear because the message immediately tells what program was added.
3	feat: add student structure	feat: add student structure	Both are equally clear because the message identifies the feature added.




ChatGPT Classic
Share
Write a commit message for the following change, using the format "type: short imperative summary" under 50 characters. Give only the message, no explanation.
Change: I added a fibonacci Python program to my repository.
feat: add fibonacci Python program
ChatGPT can make mistakes. Check important info.
Ask anything
Think
27°C
Mostly cloudy
Search
9:58 PM
9/11/2026


ChatGPT Classic
Share
Write a commit message for the following change, using the format "type: short imperative summary" under 50 characters. Give only the message, no explanation.
Change: I added a structure Python program to my repository.
feat: add structure Python program
ChatGPT can make mistakes. Check important info.
Ask anything
Think
27°C
Mostly cloudy
Search
9:59 PM
9/11/2026

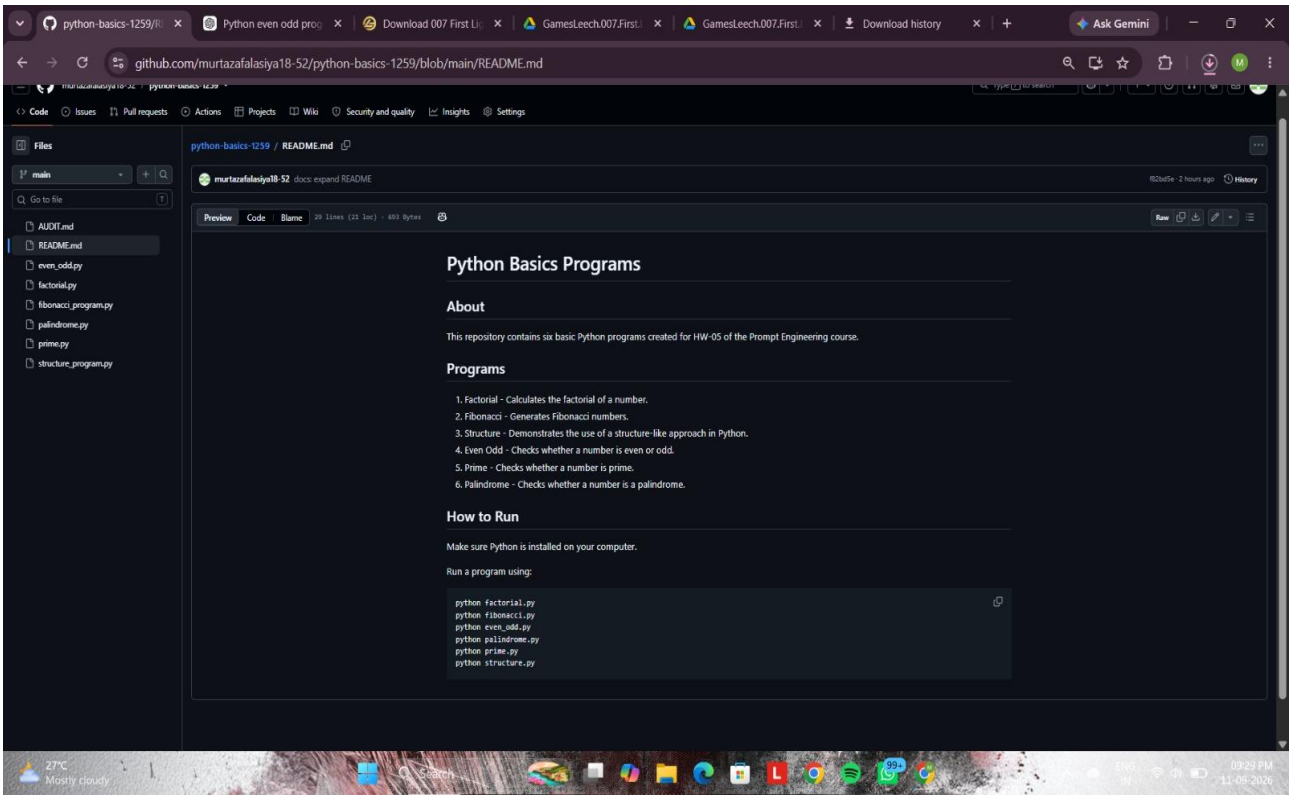
3. README: Six Programs + Claim Audit

README Content

- factorial.py — calculates the factorial of a number using a loop.
- fibonacci.py — generates a Fibonacci sequence for a requested number of terms.
- structure.py — uses a Python class to store and display student information.
- even_odd.py — determines whether an input number is even or odd.
- prime.py — checks whether an input number is prime.
- palindrome.py — checks whether an input value reads the same forward and backward.

How to Run

```
python factorial.py
python fibonacci.py
python structure.py
python even_odd.py
python prime.py
python palindrome.py
```



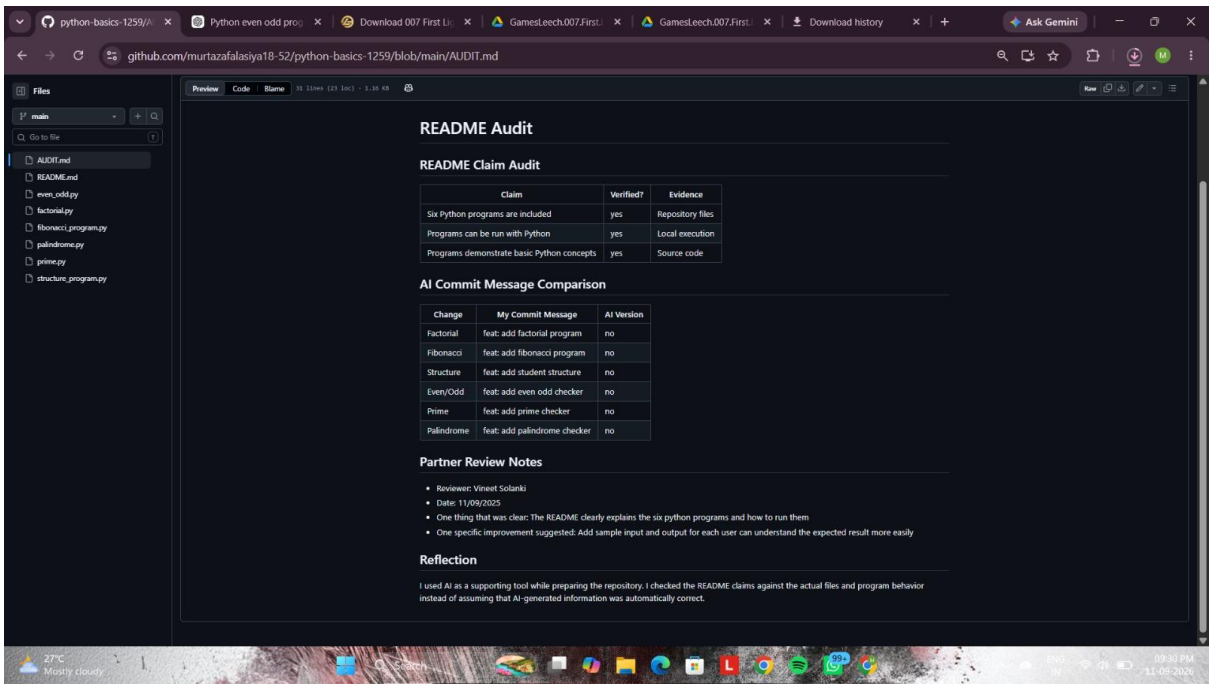
README Audit Table

Claim made in README	True?	Evidence / correction made
Six programs are present.	TRUE	Verified that factorial.py, fibonacci.py, structure.py, even_odd.py, prime.py, and palindrome.py are present in the GitHub repository.
factorial.py calculates factorial.	TRUE	Ran the program with a known input and compared the output with the expected factorial result.
fibonacci.py generates requested terms.	TRUE	Ran the program with a known input and verified the generated Fibonacci sequence.
structure.py stores student information.	TRUE	Checked the code and output to verify that student information is stored/displayed.
No external packages are required.	TRUE	Checked the imports in all six Python programs; no external packages are used.
All README commands work.	TRUE	Ran the commands listed in the README and verified that they execute successfully.

4. README Audit + Strong Public README Benchmark

Audit Mindset

The repository is the source of truth. For every README claim, check whether the named file exists, whether described behavior matches the code, whether commands work, and whether dependency or feature claims are supported.



README Audit

README Claim Audit

Claim	Verified?	Evidence
Six Python programs are included	yes	Repository files
Programs can be run with Python	yes	Local execution
Programs demonstrate basic Python concepts	yes	Source code

AI Commit Message Comparison

Change	My Commit Message	AI Version
Factorial	feat: add factorial program	no
Fibonacci	feat: add fibonacci program	no
Structure	feat: add student structure	no
Even/Odd	feat: add even odd checker	no
Prime	feat: add prime checker	no
Palindrome	feat: add palindrome checker	no

Partner Review Notes

- Reviewer: Vineet Solanki
- Date: 11/09/2025
- One thing that was clear: The README clearly explains the six python programs and how to run them
- One specific improvement suggested: Add sample input and output for each user can understand the expected result more easily

Reflection

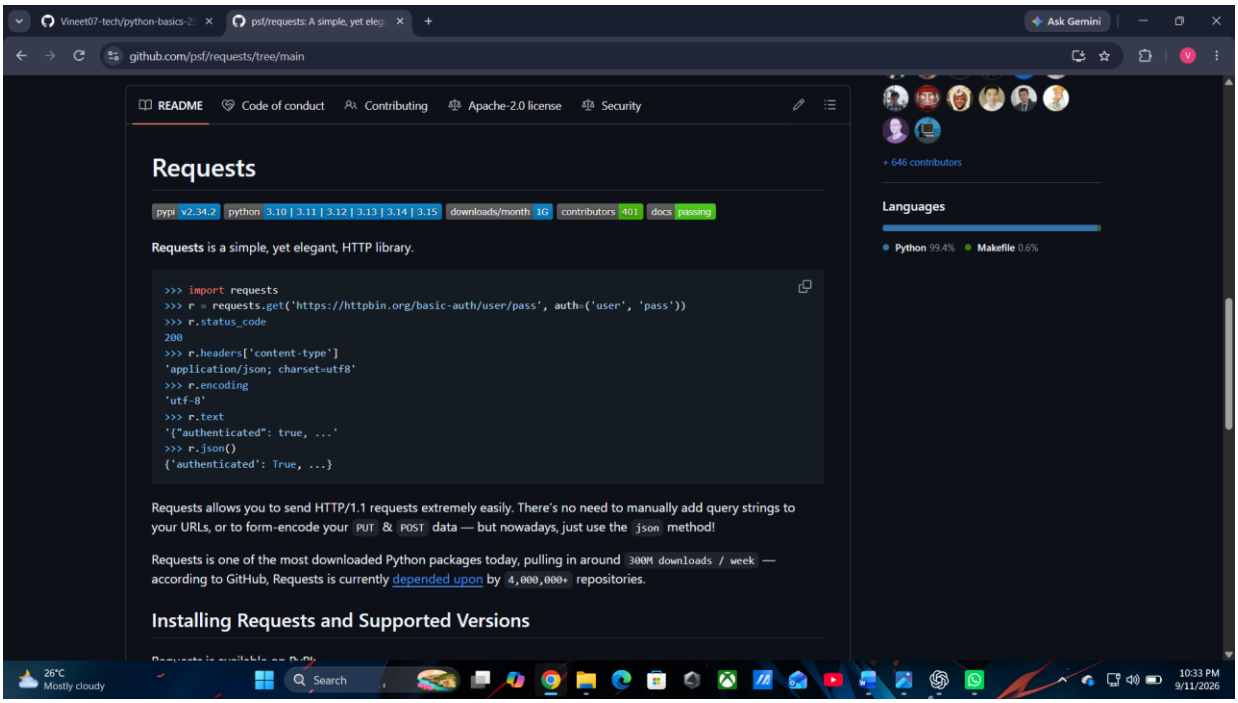
I used AI as a supporting tool while preparing the repository. I checked the README claims against the actual files and program behavior instead of assuming that AI-generated information was automatically correct.

Benchmark: psf/requests README

The public Requests README is a useful benchmark because it gives a concise project description, an installation command, supported Python versions, feature information, examples, and source/repository information. These qualities can be applied to a smaller student repository.

Benchmark source: GitHub – psf/requests README: https://github.com/psf/requests/blob/main/README.md

•



README | Code of conduct | Contributing | Apache-2.0 license | Security

Requests

pypi v2.34.2 | python 3.10 | 3.11 | 3.12 | 3.13 | 3.14 | 3.15 | downloads/month 1G | contributors 401 | docs passing

Requests is a simple, yet elegant, HTTP library.

```
>>> import requests
>>> r = requests.get('https://httpbin.org/basic-auth/user/pass', auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf8'
>>> r.encoding
'utf-8'
>>> r.text
'{"authenticated": true, ...'
>>> r.json()
{'authenticated': True, ...}
```

Requests allows you to send HTTP/1.1 requests extremely easily. There's no need to manually add query strings to your URLs, or to form-encode your `PUT` & `POST` data — but nowadays, just use the `json` method!

Requests is one of the most downloaded Python packages today, pulling in around `300M downloads / week` — according to GitHub, Requests is currently depended upon by `4,000,000+` repositories.

Installing Requests and Supported Versions

+ 646 contributors

Languages

Python 99.4% Makefile 0.6%

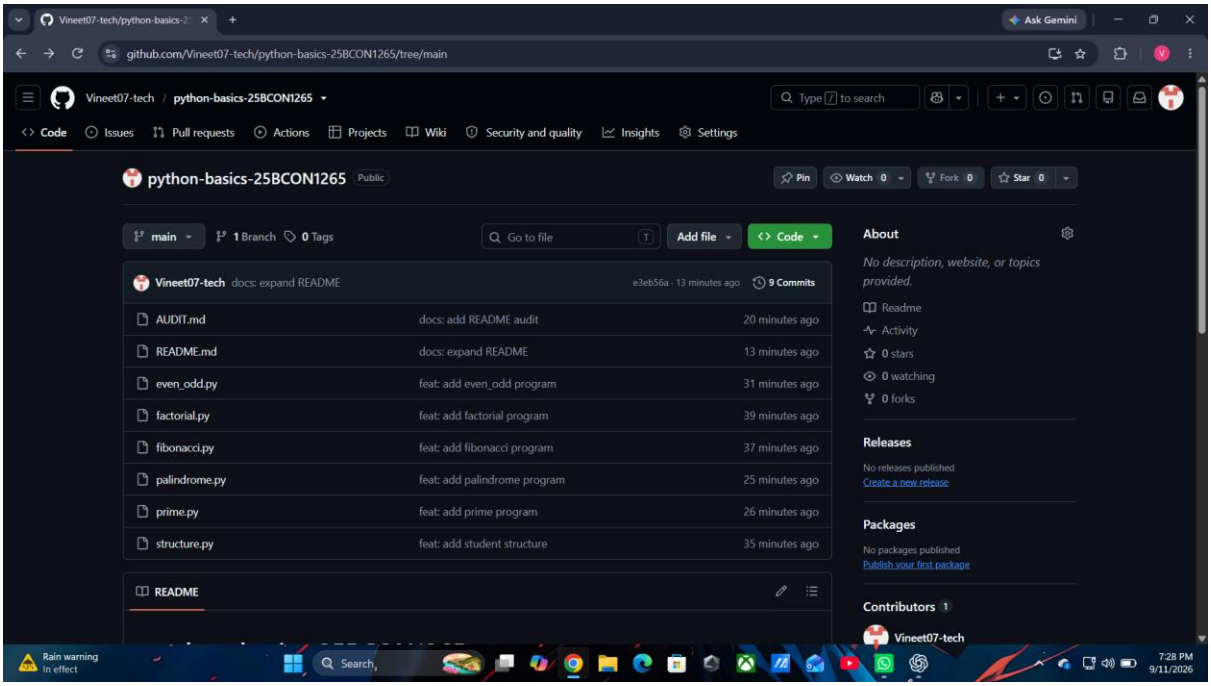
5. Peer Review + Personal Reflection

Peer Repository Review

The peer review requires exchanging public repository links, understanding the repository quickly, verifying two claims, reading commit messages, and giving one specific fix. The partner's notes should be appended to AUDIT.md.

Partner name: Vineet Solanki

Partner repository link: https://github.com/Vineet07-tech/python-basics-25BCON1265



Review item	Actual notes
Claim 1 verified	The repository contains all six Python programs listed in the README.
Claim 2 verified	The README commands match the Python filenames in the repository.
Commit history observation	The programs were added through separate commits with clear and concise commit messages.
One specific fix	Add sample input and output for each program to make the README easier to understand.

Personal Reflection

Through this assignment, I learned that a public repository is not only a place to store code but also a record of how the code was created and changed. Clear commit messages make the history easier for another person to understand.

I also learned that an AI-generated README cannot be trusted simply because it is fluent. Statements about files, dependencies, features, and commands must be checked against the actual repository. Unsupported claims should be corrected or removed.

My main lesson is that public documentation is part of professional evidence. Before publishing a README or commit message, I should be able to defend every statement in it.